AI Tooling Decisions

This section is written to match the case-study requirement exactly: for each major activity, I list `3` realistic AI-driven options or frameworks, select `1`, explain why I chose it, explain why I did not choose the other two, and tie the choice back to the goals of this POC.

The overall principle behind my choices was simple: optimize for `speed`, `clarity`, `low setup friction`, and `grounded support behavior` rather than chasing the most complex stack.

1. Requirements gathering and user story identification

Options considered

1. `GPT-5.4 in Codex`
2. `Claude Opus 4.5`
3. `Gemini 2.5 Pro`

Selected

`GPT-5.4 in Codex`

Why I chose it

- The brief is intentionally incomplete, so I needed a model that could move between product thinking, stakeholder questioning, system design, and implementation planning without losing context.
- GPT-5.4 worked well for turning a vague request into a sharper problem definition, MVP scope, assumptions, and hidden client needs.
- Using the same environment for requirements shaping and implementation reduced handoff friction and kept the submission story consistent.

Why I did not choose the other two

- `Claude Opus 4.5`: very strong for product writing and ambiguity handling, but I preferred tighter workflow continuity with the environment I used to actually build the prototype.
- `Gemini 2.5 Pro`: broad and capable, but I wanted more predictable output for requirement refinement and clearer continuity with the rest of my build workflow.

Why this aligns with the solution goals

This project required more than summarizing a brief. It required identifying what the client forgot to ask for: handoff, grounded answers, analytics, and bilingual readiness. GPT-5.4 in Codex supported that kind of systems-level framing while keeping the work close to the codebase.

2. Test case development

Options considered

1. `GPT-5.4 in Codex`
2. `Claude 4.5 Sonnet`
3. `Gemini 2.0 Flash`

Selected

`GPT-5.4 in Codex`

Why I chose it

- The most important tests in this project were not generic CRUD tests; they were conversation-flow tests with clarifications, follow-ups, Arabic/English behavior, and policy grounding.
- GPT-5.4 was effective at generating both happy-path and edge-case scenarios for multi-turn support behavior.
- It also helped keep the tests aligned with the business logic of the chatbot, not just the current implementation details.

Why I did not choose the other two

- `Claude 4.5 Sonnet`: strong quality, but I preferred keeping test authoring inside the same active repo context and toolchain.
- `Gemini 2.0 Flash`: attractive for speed and cost, but I wanted deeper reasoning for nuanced support journeys like refund clarification and bilingual continuity.

Why this aligns with the solution goals

The value of this POC depends on proving that the chatbot behaves reliably on the most important customer journeys. GPT-5.4 helped generate and refine tests that validate trust-sensitive flows such as order tracking, returns/refunds, and human escalation.

3. Code development IDE or environment

Options considered

1. `Codex CLI with VS Code`
2. `VS Code with GitHub Copilot`
3. `Cursor`

Selected

`Codex CLI with VS Code`

Why I chose it

- Codex supported an end-to-end agentic workflow across product framing, architecture, implementation, review, and documentation, with VS Code as the editing environment.

- It was especially useful for keeping multiple files aligned at once, which mattered here because the docs, prototype, and demo script all needed to tell the same story.
- It reduced context switching and helped move quickly from ambiguous requirements to a working MVP.

Why I did not choose the other two

- `VS Code with GitHub Copilot`: reliable for in-editor assistance, but weaker for managing the full workflow across docs, architecture, code, and review in one continuous loop.
- `Cursor`: strong environment for AI-assisted coding, but I preferred Codex for this specific case because of the smoother agentic workflow, better fit for repository-wide alignment tasks, and lower cost.

Why this aligns with the solution goals

This submission was not only about writing code. It was about demonstrating product judgment, fast prototyping, and systems thinking. Codex CLI with VS Code fit that requirement better than a purely code-completion-oriented setup.

4. Code review

Options considered

1. `Greptile`
2. `GitHub Copilot`
3. `GPT-5.4 in Codex`

Selected

`Greptile and GitHub Copilot`

Why I chose it

- For code review, I would select Greptile together with GitHub Copilot because they complement each other well.
- Greptile is strong for repository-level understanding, codebase-aware review, and identifying implementation issues in the broader project context.
- GitHub Copilot is useful for fast in-editor review support, spotting local improvements, and accelerating iteration while refining code.
- Together, they provide both high-level repository understanding and practical developer-speed feedback.

Why this aligns with the solution goals

This POC needed review at two levels: correctness of the code and consistency across the repository. Greptile fits the repository-wide review need, while GitHub Copilot supports

fast iteration inside the editor. That combination is well suited for catching support-flow issues, integration gaps, and implementation inconsistencies in a fast-moving prototype.

5. Manual tests, automated tests, and unit tests

Options considered

1. `GPT-5.4 in Codex`
2. `Playwright + GPT-4.1`
3. `Cypress + Claude 3.5 Sonnet`

Selected

`GPT-5.4 in Codex`

Why I chose it

- GPT-5.4 in Codex was used to generate, refine, and strengthen the unit tests for this POC.
- The most important testing need in this project was validating chatbot behavior across multi-turn support flows such as product information, order tracking, returns/refunds, bilingual behavior, and handoff logic.
- Keeping test development inside Codex reduced context switching and helped ensure that the tests stayed closely aligned with the implementation and the business goals of the chatbot.

Why I did not choose the other two

- `Playwright + GPT-4.1`: strong choice for full browser automation, but heavier than necessary for this MVP and less aligned with the biggest project risk, which was conversation correctness rather than deep browser compatibility.
- `Cypress + Claude 3.5 Sonnet`: also a valid UI-testing setup, but it would have added setup and maintenance overhead that was hard to justify for a compact POC.

Why this aligns with the solution goals

The goal of this case study is to demonstrate fast prototyping, good judgment, and a working AI-first support MVP. GPT-5.4 in Codex supported that by helping build automated tests around the flows that matter most for trust and customer experience: support accuracy, escalation handling, and multilingual continuity.

Summary of actual selections

- Requirements gathering and user story identification: `GPT-5.4 in Codex`
- Test case development: `GPT-5.4 in Codex`

- Code development IDE/environment: `Codex CLI with VS Code`
- Code review: `Greptile and GitHub Copilot`
- Manual tests, automated tests, unit tests: `GPT-5.4 in Codex`

Why this overall stack made sense for this POC

The selected stack optimized for:

- `speed`: fast iteration from ambiguous brief to working prototype
- `clarity`: strong support for writing the problem definition, solution design, and demo narrative
- `cost and setup`: light dependencies and easy reviewer setup
- `trust`: grounded product, order, and policy answers instead of free-form support hallucinations
- `scalability`: a prototype structure that can later connect to commerce, CRM, courier, and analytics systems

For this case study, that trade-off was more valuable than building the heaviest or most automated stack possible.